```
  retry if e.okay_to_retry
  raise
end
```

Using Catch and Throw

Although the exception mechanism of raise and rescue is great for abandoning execution when things go wrong, it's sometimes nice to be able to jump out of some deeply nested construct during normal processing. This is where the rarely used catch and throw come in handy.

Here's a trivial example. The following code reads a list of words one at a time and adds them to an array. When done, it prints the array in reverse order. However, if any of the lines in the file doesn't contain a valid word, we want it to abandon the whole process.

```
word_list = File.open("wordlist")
catch (:done)  do
  result = []
  while line = word_list.gets
    word = line.chomp
    throw :done unless word =~ /^\w+$/
    result << word
  end
  puts result.reverse
end
```

catch defines a block that is labeled with the given name (which may be a Symbol or a String). The block is executed normally until a throw is encountered.

When Ruby encounters a throw, it zips back up the call stack looking for a catch block with a matching symbol. When it finds it, Ruby unwinds the stack to that point and terminates the block. So, in the previous example, if the input does not contain correctly formatted lines, the throw will skip to the end of the corresponding catch, not only terminating the while loop but also skipping the code that writes the reversed list. If the throw is called with the optional second parameter, that value is returned as the value of the catch. In this example, our word list incorrectly contains the line *wow*. Without the second parameter to throw, the corresponding catch returns nil.

```
word_list = File.open("wordlist")
word_in_error = catch(:done)  do
  result = []
  while line = word_list.gets
    word = line.chomp
    throw(:done, word) unless word =~ /^\w+$/
    result << word
  end
  puts result.reverse
end
if word_in_error
  puts "Failed: '#{word_in_error}' found, but a word was expected"
end
```

produces:

```
Failed: '*wow*' found, but a word was expected
```